



UNIVERSITÀ
degli STUDI
di CATANIA

DIPARTIMENTO DI
INGEGNERIA ELETTRICA ELETTRONICA ED INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

RELAZIONE TECNICA

FOOD BLOGGER COPILOT

Alunni:

Arena Giorgia

Scarvaglieri Antonio Nicolò

Docenti:

Giordano Daniela

Salvatore Calcagno

ANNO ACCADEMICO 2025 – 2026

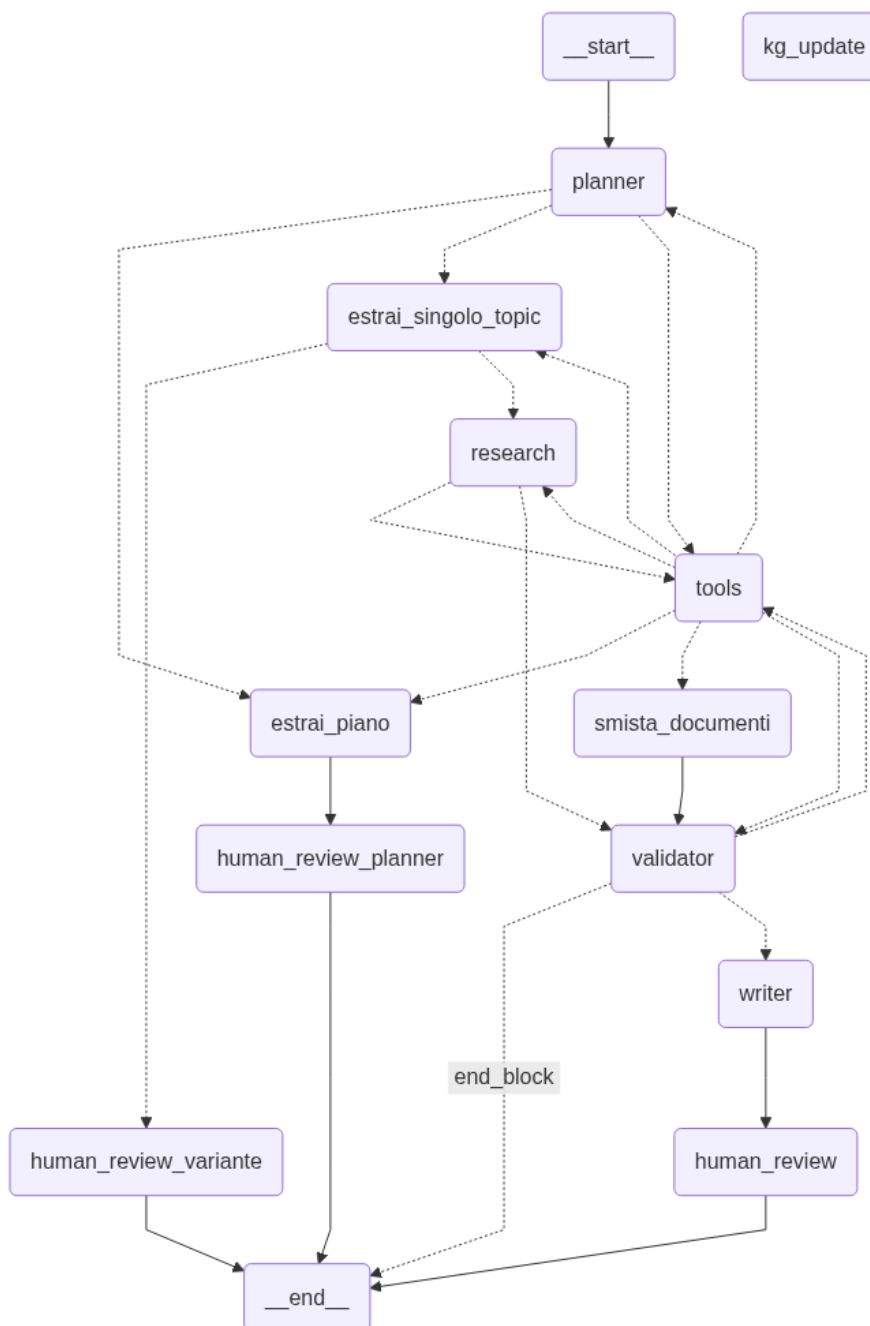
INDICE

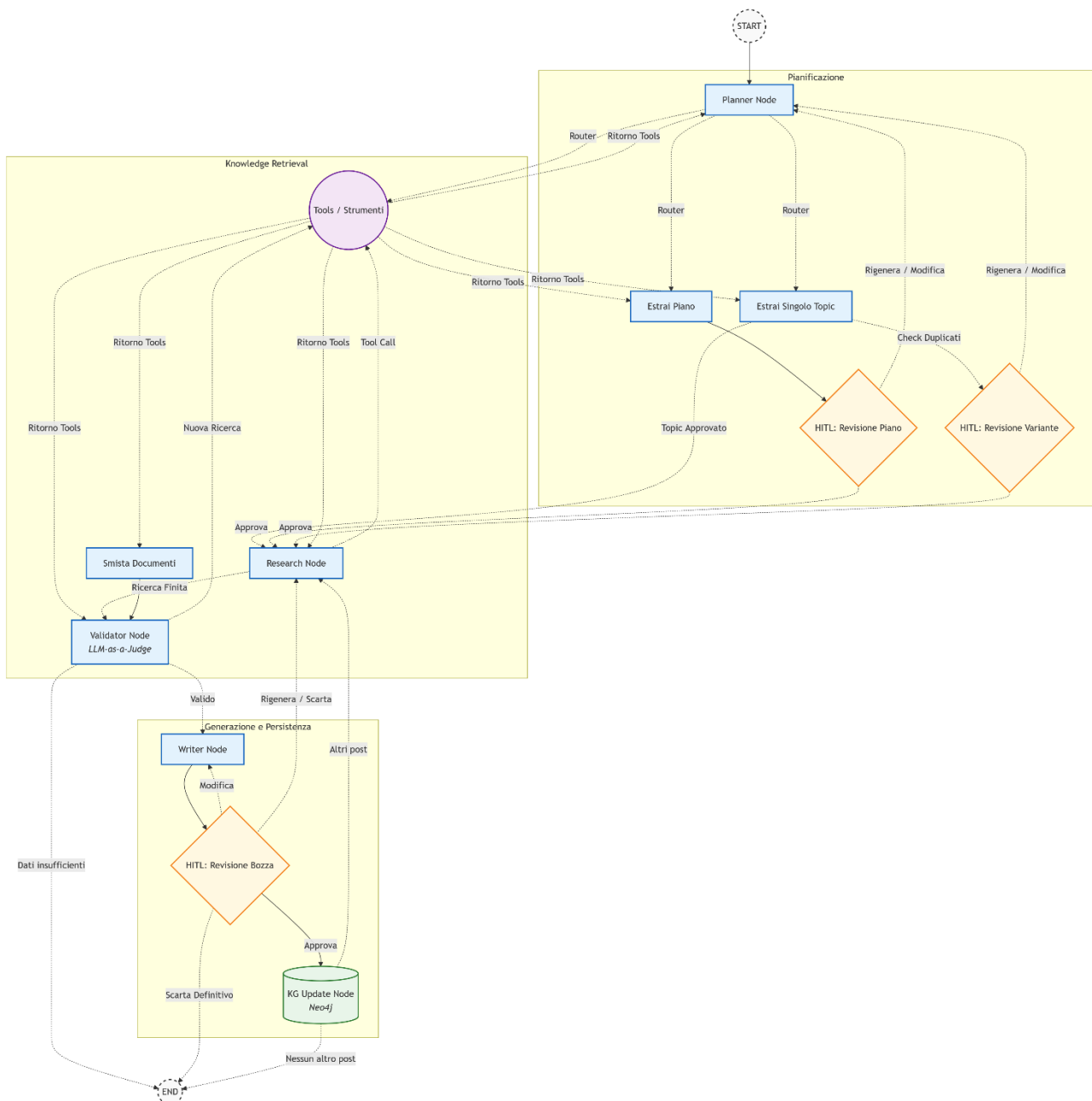
1.	DESIGN CHOICES OF ARCHITECTURE	3
1.1	PANORAMICA ARCHITETTURALE E CORE AGENTICO (LANGGRAPH)	3
1.2	ARCHITETTURA RAG IBRIDA: VECTOR RETRIEVAL E GRAPHRAG	6
1.2	IL TOOL CUSTOM PER L'INFORMATION EXTRACTION (MODELLO FINE-TUNED).....	9
2.	PROBLEMS ENCOUNTERED AND SOLUTIONS	10
2.1	PROBLEMA ARCHITETTURALE: DATA CORRUPTION E IL "MERGE CONFUSO" SU NEO4J	10
2.2	PROBLEMA DI MACHINE LEARNING: TRAINING-SERVING SKEW E BIAS CONSERVATIVO	ERRORE. IL SEGNALIBRO NON È DEFINITO.
3.	EXPERIMENTAL AND EVALUATION ACTIVITIES	11
3.1	METODOLOGIA DI TRAINING (PEFT) E HYPERPARAMETER SEARCH	ERRORE. IL SEGNALIBRO NON È DEFINITO.
3.2	EVALUATION: BASELINE VS FINE-TUNED.....	ERRORE. IL SEGNALIBRO NON È DEFINITO.
4.	CONCLUSIONI E SVILUPPI FUTURI.....	12

1. DESIGN CHOICES OF ARCHITECTURE

1.1 Panoramica Architetture e Core Agentico (LangGraph)

L'infrastruttura di orchestrazione del nostro *Food Blogger Copilot* abbandona il paradigma lineare (tipico delle chain standard) a favore di un workflow Multi-Agente ciclico e *stateful*, implementato tramite la libreria **LangGraph**.





La scelta di LangGraph come core agentico è stata dettata da due requisiti architetturali imprescindibili:

1. **Gestione di cicli di revisione:** Necessità di far tornare indietro il flusso di esecuzione (routing ciclico) nel caso in cui l'umano richieda modifiche.
2. **Mantenimento di uno Stato Globale:** L'agente deve ricordare il contesto attraverso molteplici chiamate LLM indipendenti, accumulando informazioni passo dopo passo prima di procedere alla stesura finale.

Il workflow si articola attraverso la gestione di un oggetto State condiviso e una serie di nodi funzionali operando in logica di "Separation of Concerns":

- **Gestione dello Stato (State Management):** Lo stato del grafo funge da memoria a breve termine. Raccoglie la query originale, la ricetta_madre recuperata, la lista delle sottoricette identificate con i relativi ingredienti, e infine il draft_post. Questo previene la saturazione della *Context Window* dei modelli, passando a ogni nodo solo i parametri di cui ha bisogno.
- **Nodo Planner:** E' il nodo fondamentale da cui parte il nostro flusso. Si suddivide in pianificazione automatica e pianificazione di un singolo post. Il planner si occuperà attraverso i nodi intermedi di estrazione ad estrarre i singoli topic o il piano editoriale proposto. Il planner proporrà varianti e piani editoriali in base alla propria conoscenza interna(claim e post) dal KG.
- **Nodo di Ricerca ed Estrazione (Research & Test della Sottoricetta):** È il motore di acquisizione della conoscenza. Ricevuta la query dell'utente, interroga il Vector DB (ChromaDB) per recuperare il testo della "Ricetta Madre". Immediatamente dopo, per evitare allucinazioni e fallimenti di parsing, il testo grezzo della ricetta non viene passato a un LLM generalista, ma viene analizzato dal nostro tool custom: il modello **Llama-3.2-1B Fine-Tuned via QLoRA**. Questo modello agisce come estrattore deterministico, restituendo un array JSON con le sole sottoricette presenti nel testo della ricetta madre.
- **HITL post e HITL Piano:** Sia il singolo topic(incluse varinati) che il piano editoriale , verranno sempre sottoposti a revisione umana. L'utente potrà rigenerare/ modificare/approvare il piano o il singolo topic.
- **Nodo Validatore (Evaluation & Filtering):** Prima di procedere alla stesura, i documenti recuperati attraversano un nodo di validazione. Questo componente agisce come filtro di affidabilità applicando il pattern **LLM-as-a-Judge**: un LLM valuta la pertinenza semantica delle fonti rispetto alla query originale e assegna uno score di qualità, scartando contestualmente i duplicati, i contenuti off-topic o i siti non attendibili. Ciò garantisce che la sintesi finale avvenga solo su un contesto pulito e altamente rilevante.

- **Nodo di Stesura (Writer):** Solo a questo punto, con lo Stato del grafo pienamente popolato, viene invocato un LLM generalista. Il suo compito non è la mera trascrizione, ma una sintesi discorsiva guidata da un rigoroso **Prompt Engineering**. Tramite l'iniezione dinamica delle variabili di Stato (la ricetta validata, le sottoricette e i *claim* dell'utente), il prompt istruisce il modello a redigere la bozza finale adottando il *Tone of Voice* tipico di un "Food Blogger". Questo garantisce il rispetto di eventuali vincoli stilistici senza mai alterare le dosi, i passaggi e gli ingredienti consolidati nelle fasi precedenti.
- **HITL Post:** È il punto critico di sicurezza dell'architettura. Il grafo sospende l'esecuzione (*Interrupt*) e restituisce la bozza e i dati strutturati all'interfaccia utente. Da qui, un *Conditional Edge* valuta l'input umano:
 - o Se l'utente richiede modifiche, il grafo ruota all'indietro verso il Nodo di Stesura.
 - o Se l'utente approva, il flusso transita verso il nodo finale di persistenza su Neo4j.
 - o L'utente può anche rigenerare il post effettuando nuovamente il flusso.

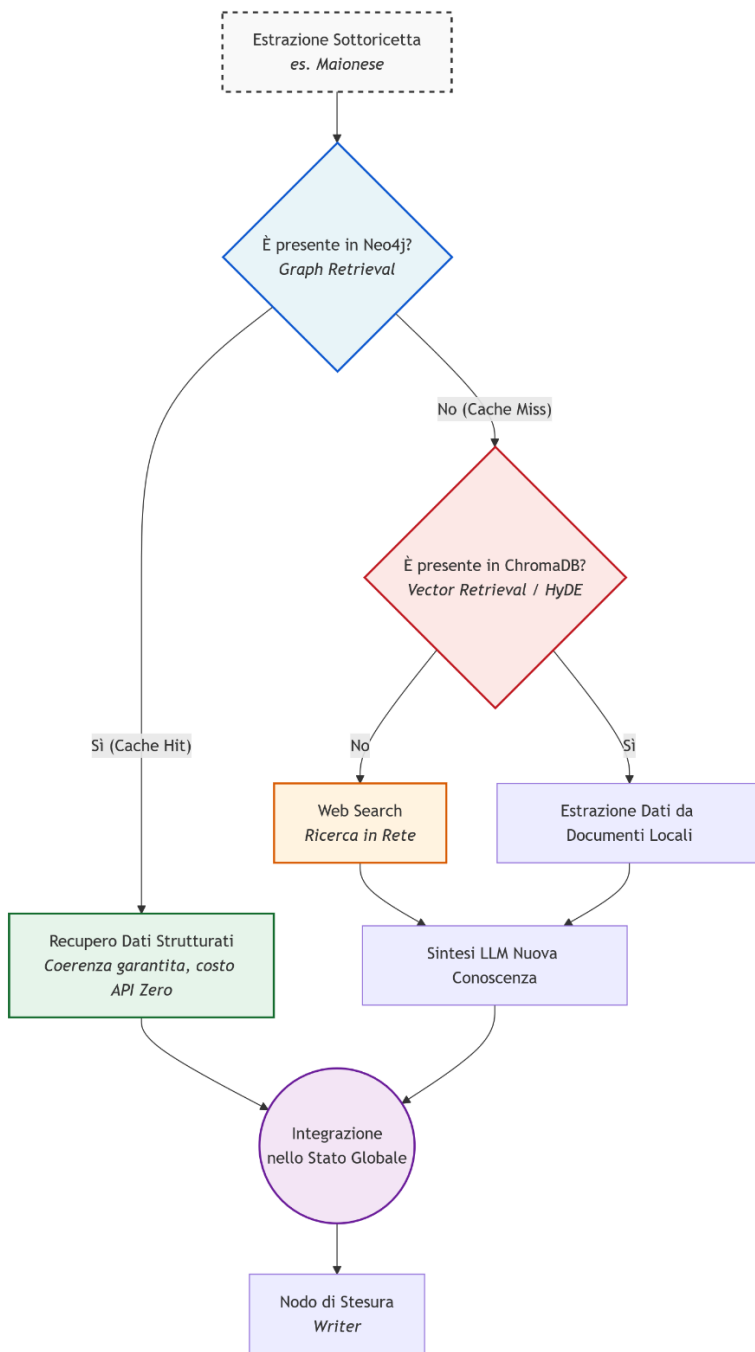
Questa implementazione garantisce che l'Intelligenza Artificiale operi strettamente come "Copilot": il motore cerca, estrae e struttura la conoscenza, ma **nessun dato viene consolidato in modo persistente nel Knowledge Graph senza un'esplicita validazione umana**, proteggendo il database da corruzioni autonome.

1.2 Architettura RAG Ibrida: Vector Retrieval e GraphRAG

Per soddisfare i requisiti di coerenza enciclopedica (Consistency) e robustezza richiesti dal dominio culinario, abbiamo superato i limiti del RAG (Retrieval-Augmented Generation) tradizionale, implementando una pipeline di **Modular Krag (Knowledge-RAG) a doppio strato**.

Il sistema assegna responsabilità semantiche distinte a due database differenti, unendo la flessibilità della ricerca vettoriale all'affidabilità dei database a grafo.

[SEGUE IMMAGINE]



Questa implementazione garantisce che la pipeline KRAG sia al contempo reattiva verso le nuove richieste (grazie ai vettori e al web) ed enciclopedicamente coerente con il proprio passato (grazie al grafo).

1.2 Il Tool Custom per l'Information Extraction (Modello Fine-Tuned)

Nel design di architetture Agentic AI, delegare task di estrazione strutturata rigorosa a un Large Language Model generalista (tramite Prompt Engineering Zero-Shot) introduce due criticità architetturali: un elevato tasso di fallimento nel parsing (dovuto alla naturale verbosità del modello, che tende ad aggiungere preamboli discorsivi invalidando i formati JSON) e una scarsa precisione semantica (incapacità di discernere casi limite).

Per risolvere queste problematiche, abbiamo disaccoppiato la fase di sintesi testuale dalla fase di analisi, ingegnerizzando un **Custom Node** dedicato esclusivamente all'Information Extraction. Questo nodo non utilizza il modello di base, ma invoca un modello specializzato, ottenuto tramite Fine-Tuning di **Llama-3.2-1B-Instruct**. Le scelte ingegneristiche per lo sviluppo di questo componente si dividono in due filoni principali:

- **Approccio Data-Centric (Il ruolo degli Hard Negatives):** Abbiamo addestrato il modello su un dataset custom di 250 esempi formattati in standard ChatML, fornendogli come input l'intero testo "sporco" della "Ricetta Madre" (esattamente come lo riceverà in produzione dal nodo di ricerca) e come target l'array JSON atteso. La vera innovazione risiede nella costruzione del dataset: per impedire al modello di imparare un banale *pattern matching* sui verbi culinari, abbiamo deliberatamente iniettato una forte percentuale di **Hard Negatives**. Il modello è stato addestrato a restituire un array vuoto [] in presenza di ingredienti pronti, preparazioni a crudo o tagli base. Questo ha forzato il meccanismo di *Self-Attention* a interiorizzare i complessi confini logici tra un semplice ingrediente e una preparazione autonoma.

- **Approccio Model-Centric (Parameter-Efficient Fine-Tuning):** Per rendere l'addestramento computazionalmente sostenibile e prevenire il *catastrophic forgetting* (la perdita delle capacità linguistiche originarie dell'LLM), abbiamo scartato il Full Fine-Tuning a favore della tecnica **QLoRA**. Il modello base è stato quantizzato a 4-bit, permettendoci di addestrare esclusivamente degli adattatori LoRA (Low-Rank Adaptation) applicati ai layer di attenzione lineari (q_proj, v_proj, k_proj, o_proj). Questo ha consentito di ottimizzare il task con un'efficienza architetturale estrema, **aggiornando appena lo 0.45% dei parametri totali** del modello, mantenendo l'occupazione della VRAM ampiamente entro i limiti operativi di una singola GPU.

Il risultato di questa pipeline è la trasformazione di un LLM conversazionale in un "Tool" software deterministico: riceve in input testi destrutturati recuperati dal RAG e restituisce, con una *Format Compliance* assoluta del 100%, un array JSON perfetto, abbattendo drasticamente il tasso di allucinazione e pilotando in sicurezza il routing del nodo successivo.

2. PROBLEMS ENCOUNTERED AND SOLUTIONS

Durante la fase di system testing e integrazione della pipeline distribuita, sono emerse criticità relative alla persistenza dei dati nel database a grafo e al comportamento predittivo del modello Fine-Tuned. Di seguito analizziamo i problemi e i pattern ingegneristici adottati per la loro risoluzione.

2.1 Problema Architetturale: Data Corruption e il "Merge Confuso" su Neo4j

- **Analisi del Problema:** In un'architettura incrementale, la gestione delle scritture sovrapposte è critica. Inizialmente, il salvataggio dei post sul Knowledge Graph (Neo4j) era affidato all'operatore Cypher MERGE. Tuttavia, durante i test del *Multi-hop Retrieval*, è emerso un limite strutturale grave: se il blog aveva già validato tramite HITL e salvato una sottoricetta (es. *Maionese*), un post

successivo (es. *Insalata Russa*) avrebbe potuto innescare un fallback su ChromaDB/Web per la medesima preparazione. Se l'agente avesse estratto una lista di ingredienti leggermente diversa da questa nuova fonte, l'istruzione MERGE avrebbe sovrascritto silenziosamente gli archi CONTIENE del nodo originale. Il risultato era un nodo "Frankenstein" (dati duplicati o incoerenti), che distruggeva il principio della *Single Source of Truth*.

- **Soluzione Ingegneristica** Se il nodo della sottoricetta possiede già archi di tipo CONTIENE in uscita, il sistema lo identifica come "Entità Validata" e blocca l'operazione di ricerca. Questo garantisce che la prima versione approvata dall'operatore umano diventi la baseline immutabile del blog, isolando il Knowledge Graph da eventuali allucinazioni o varianti introdotte in esecuzioni successive.

3. EVALUATION LANGSMITH

3.1 Valutazione della Qualità (Quality Score)

Il sistema valuta la bontà del testo generato affidandosi a un giudice IA (GPT-4o-mini) istruito per comportarsi come un severo critico gastronomico. L'agente legge la bozza finale del post e assegna un punteggio da 1 a 5 basandosi su parametri strettamente editoriali e tecnici:

- Chiarezza espositiva: La struttura del post deve essere logica e facile da seguire.
- Stile e Tono: Il testo deve essere coinvolgente per il lettore e completamente privo di ripetizioni fastidiose.
- Precisione Tecnica: Le dosi degli ingredienti e i passaggi della ricetta devono avere senso e risultare realistici (penalizzando eventuali "invenzioni" assurde sulle quantità).

Il voto verrà espresso in maniera percentuale.

3.2 Analisi dei Casi di Fallimento (Failure Cases),

Questa metrica è un "controllo di sicurezza" matematico che agisce come un filtro anti-errore. Il sistema parte da un punteggio perfetto e sottrae un quarto di punto (0.25) per ognuna di queste 4 infrazioni critiche commesse dall'agente:

- F1 - Bozza Vuota (Empty Draft): L'agente ha concluso l'esecuzione ma non ha prodotto alcun testo.
- F2 - Mancanza di Citazioni (No Citations): Il testo è stato scritto, ma l'agente ha dimenticato di inserire all'interno dell'articolo i marcatori per la bibliografia (es. "fonti utilizzate:" o link web).
- F3 - Assenza di Fonti (No Sources): L'agente ha scritto il post basandosi solo su se stesso, senza prima interrogare e salvare le fonti necessarie dal Database Vettoriale o dalla ricerca Web.
- F4 - Blocco Anomalo (No Interrupt): L'agente si è fermato improvvisamente a causa di un errore interno o di un loop, senza arrivare a chiedere la tua approvazione manuale (i famosi "nodi di review").

4. CONCLUSIONI E SVILUPPI FUTURI

L'architettura proposta dimostra come l'integrazione di LLM generativi in domini strutturati richieda un rigoroso controllo ingegneristico. Affidare la coerenza del sistema a una singola tecnica (come il Prompt Engineering o il Vector RAG) si è rivelato insufficiente. Solo orchestrando pattern complessi — come il **Multi-Agente stateful** basato su ReAct, l'**Adaptive GraphRAG** per la tutela della Single Source of Truth contro i merge confusi, e il **Parameter-Efficient Fine-Tuning** per abbattere il tasso di allucinazione nei task deterministici — siamo riusciti a costruire un sistema robusto, capace di automatizzare la stesura di contenuti culinari garantendo al contempo infallibilità enciclopedica e assenza di allucinazioni.

Come sviluppi futuri, l'architettura si presta naturalmente all'espansione del Knowledge Graph con nuovi strati semantici (es. tracciamento degli allergeni o calcolo

del profilo nutrizionale), che l'agente potrebbe interrogare per generare post sempre più iper-personalizzati sulle esigenze dietetiche dell'utente.

Il progetto è reperibile al seguente link: <https://github.com/assasinghira01/Progetto-agenti-2026>